

TORONTO METROPOLITAN UNIVERSITY

CIND 830 PYTHON PROGRAMMING FOR DATA SCIENCE

Meal Planner and Shopping List Generator

Individual Assignment

Introduction

This individual assignment focuses on building an interactive Python program that relies on menu-driven programming, repetition, and selection statements. By the end of this assignment, you will have a functional meal planner capable of scraping recipe data, generating shopping lists, and performing statistical calculations on ingredient information.

We will be using various Python libraries, each cited with its primary source:

- `recipe-scrappers` (<https://pypi.org/project/recipe-scrappers/>)
- `termcolor` (<https://pypi.org/project/termcolor/>)
- `tabulate` (<https://pypi.org/project/tabulate/>)

You can also obtain several recipes from online platforms like AllRecipes.com (<https://www.allrecipes.com/>). You can also include a special family recipe or a personal favorite that is not available online to expand your dataset and explore a variety of ingredient types.

Assignment Workflow

This assignment consists of two phases:

- **Create Phase:** Design and implement the program based on the provided requirements. Submit your solution by the deadline.
- **Assess Phase:** Evaluate the submissions of two peers using the provided rubric. Provide constructive feedback based on your findings.

Setup Instructions

1. Open your Python development environment and create a new Jupyter Notebook file (`.ipynb`). Save the file with an appropriate name (e.g., `FName_LName_Meal_Planner.ipynb`).
2. Install the required libraries by running the following command in a code cell:

```
# Install the required libraries
!pip install recipe-scrappers termcolor tabulate
```

3. Test the libraries with the snippet below:

```
from urllib.request import urlopen
from recipe_scrappers import scrape_html
from termcolor import colored

url = 'https://www.allrecipes.com/recipe/25189/chicken-alfredo-pizza/'

try:
    # Retrieve the recipe webpage HTML
    html = urlopen(url).read().decode("utf-8")

    # Pass the HTML and URL to scrape_html
    scraper = scrape_html(html, org_url=url)

    # Extract and display recipe information
    print("Title:", scraper.title())
    print("Ingredients:", scraper.ingredients())
    print("Total Time:", scraper.total_time())
    print("Yields:", scraper.yields())
    print("Instructions:", scraper.instructions())
    print(colored("Test Complete", "green"))

except ImportError as e:
    print(colored("Error: Unable to import scrape_html.", "red"))
except Exception as e:
    print(colored("An error occurred: " + str(e), "red"))
```

4. Confirm that the script displays the title of the recipe and a success message indicating the test is complete. If errors occur, recheck library installations and ensure the URL is accessible.

Assignment Description

Your program must:

1. Use the `recipe-scrappers` library to collect recipe details (title, ingredients, instructions) from at least five URLs.
2. Present an interactive menu with options to view recipes, add recipes to a meal plan, generate a shopping list, and compute statistics.
3. Merge ingredients from selected recipes into a unified list.
4. Compute the average and median number of ingredients across chosen recipes.
5. Use `tabulate` for tabular presentation and `termcolor` to colorize output for clarity.

Menu Structure

Your program should repeatedly display the following menu until the user opts to exit:

```
Menu:
1. View Recipes
2. Add Recipe to Meal Plan
3. View Meal Plan
4. Generate Shopping List
5. Calculate Statistics
6. Exit
```

Detailed Steps

Step 1: Scrape Recipes

Use `recipe-scrappers` to scrape at least five recipes. Store the scraped data in a list of dictionaries or a list of lists:

```
from recipe_scrappers import scrape_html
from urllib.request import urlopen

recipes = []

# Example recipe scraping
url = 'https://www.allrecipes.com/recipe/25189/chicken-alfredo-pizza/'
html = urlopen(url).read().decode("utf-8")
scraper = scrape_html(html, org_url=url)

recipes.append({
    "title": scraper.title(),
    "ingredients": scraper.ingredients(),
    "instructions": scraper.instructions()
})
print("Recipes loaded!")
```

Step 2: Special Recipes

Include at least one custom recipe that is not publicly available online (e.g., a family tradition or personal favorite). Manually input the title, ingredients, and instructions into your data structure.

Step 3: Menu and User Interaction

Use a `while` loop and `if-elif-else` statements to implement the menu options:

- View available recipes.
- Add recipes to a meal plan.
- Display the current meal plan.
- Generate a combined shopping list.

Step 4: Generate Shopping List

Use `collections.Counter` to combine ingredients from recipes in the meal plan:

```
from collections import Counter
from tabulate import tabulate

shopping_list = Counter()
for recipe in meal_plan:
    for ingredient in recipe["ingredients"]:
        shopping_list[ingredient] += 1

print(tabulate(shopping_list.items(),
               headers=["Ingredient", "Quantity"],
               tablefmt="grid"))
```

Step 5: Calculate Statistics

Use the `statistics` module to find the average and median number of ingredients across the selected recipes:

```
import statistics

ingredient_counts = [len(recipe["ingredients"]) for recipe in meal_plan]

if ingredient_counts:
    print("Average ingredients:", statistics.mean(ingredient_counts))
    print("Median ingredients:", statistics.median(ingredient_counts))
else:
    print("No recipes in meal plan to calculate statistics.")
```

Step 6: Colorize Output

Use `termcolor` to improve readability:

```
from termcolor import colored

print(colored("Shopping List", "blue"))
for item, qty in shopping_list.items():
    print(colored(item, "green"), ":", colored(qty, "blue"))
```

Submission Instructions

1. Submit a single Jupyter Notebook (.ipynb file) via the peerScholar platform.
2. Ensure your program runs without errors and includes code comments.
3. Provide a flowchart or control flow diagram illustrating your program's logic.
4. Add a brief section titled **Reflections on Work** (in a Markdown cell) to summarize challenges you encountered, such as broken or inconsistent recipe data, and how you addressed them.

5. Name your submission file `FirstName_LastName_MealPlanner.ipynb` and confirm that all code cells are executed and outputs are visible.
6. **Peer Code Critique:** After submission, you will be assigned two classmates' notebooks to review. Run their code locally and provide a short critique (at least 200 words) that includes at least:
 - (a) One strength of their code design.
 - (b) One suggestion for improving error handling.
 - (c) One observation on code readability.

Grading Rubric

Criteria	Points
<i>Correct and Functional Menu Implementation:</i> Clear navigation between menu options, proper user input handling, and error management.	15
<i>View, Add, and Display Meal Plan Functionality:</i> Accurate implementation of viewing recipes, adding them to the meal plan, and displaying the meal plan content.	20
<i>Shopping List Generation:</i> Proper merging of ingredients from recipes, use of <code>Counter</code> , and formatted tabular output.	15
<i>Statistical Calculations:</i> Accurate computation of average and median ingredient counts with appropriate error handling.	10
<i>Use of <code>tabulate</code> and <code>termcolor</code>:</i> Clear and visually appealing formatting and use of color for enhanced readability.	10
<i>Code Readability and Comments:</i> Well-structured code with meaningful variable names, modular design, and comprehensive comments.	10
<i>Peer Review - Code Design and Functionality:</i> Thoughtful critique of peer submissions focusing on strengths and areas for improvement.	10
<i>Peer Review - Error Handling and Readability:</i> Detailed suggestions for improving error handling and enhancing code readability.	5
<i>Submission Completion:</i> Includes all required components: code, reflections, flowchart, and successful peer reviews.	5
Total	100

You can begin this assignment by navigating to the “Assignments” section under the Table of Contents in your course shell and selecting the “PeerScholar Individual Assignment.” For additional guidance, consult the *Student Guide* available at:

<https://www.torontomu.ca/courses/toolbox/peerscholar/student-guide>

End of assignment